Lab 3 Delays

AND 9/7/26

Purpose:

Measure PIC16F883 instruction timing, use instruction-cycle calculations to create precise soft ware delays, extend those delays with nested loops, and then refactor the

Standards and references:

- See pg 8 for references/standards and rosstimo Lab 3 github
- DLG7137 Data sheet

Equipment & materials:

- MpLAB X IDE and pic-as toolchain
- PICkit programer / debugger
- PIC16F883 circuit from lab 2
- 4MHz crystal oscillator circuit
- DLG7137 Display
- Oscilloscope
- Frequency counter
- Logic Analyzer, optional
- Digital Multimeter
- Lab book

Part 1 - Measure Instruction Timing:

Goal: Use the portB counter program to measure the exicution time of PIC16F883 instructions
Use the RB0 output waveform to:

- Measure Pulse width(PW), Pulse space (PS), period, & frequency
- relate the measured waveform to the instructions exicuting in the program loop
- determine the actual instruction-cycle time, Tcy
- calculate the actual oscillator frequency
- verify the effect of adding and removing NOP instructions

AND 9/7/26

Lab 3 Delays

AWD 9/8/26

Before Lab:

Start with the working PORTB counter from Lab 2. Review the instructions executed repeatedly in the main loop program. (See pgs 12-16)

For each instruction in the repeating path:

1. identify how many instruction cycles it requires
2. determine how many instruction cycles occur

The PIC instruction clock is related to the oscillator by:

$T_{OSC} = 1/F_{OSC}$ $T_{CY} = 4T_{OSC}$ $F_{OSC} = 4/T_{CY}$

Determine the expected number of instruction cycles for:

- RB0 PW
- RB0 PS
- One complete RB0 period

For my code on pg 15 INCF takes one instruction cycle & goto takes two so for one pulse width it will take 3 instruction cycles and using a 4MHz crystal:

$T_{OSC} = 1/F_{OSC}$	$T_{CY} = 4T_{OSC}$	So RB0 PW = 3μs
$T_{OSC} = 1/4MHz$	$T_{CY} = 4 \cdot 250ns$	RB0 PS = 3μs
$T_{OSC} = 250ns$	$T_{CY} = 1\mu s$	RB0 Period = 6μs

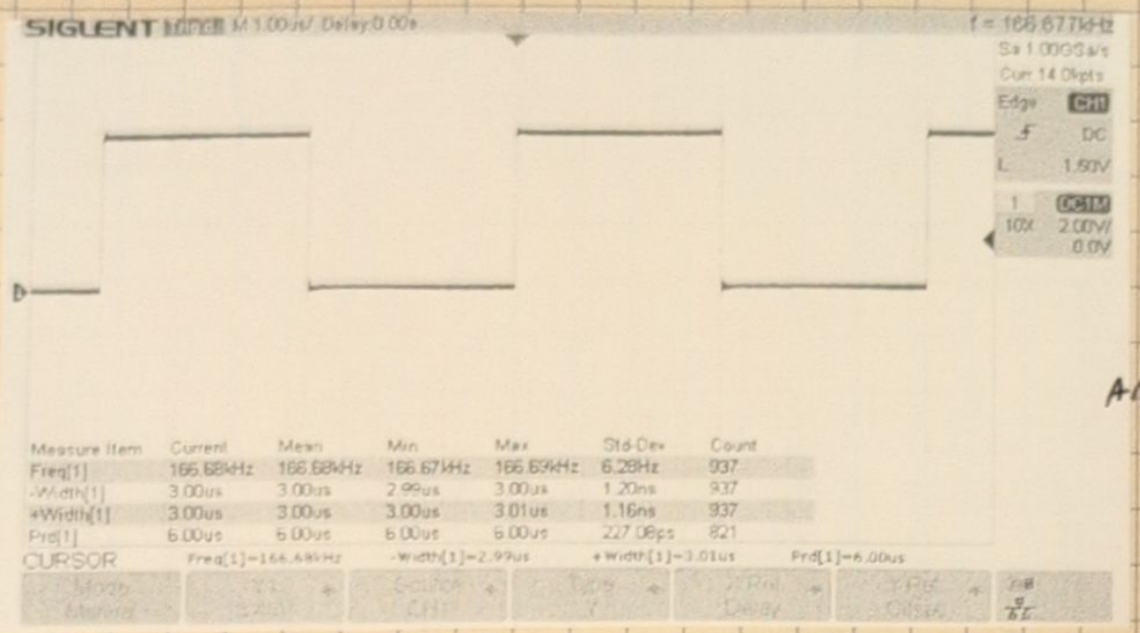
In the Lab:

1. Measure the Lab 2 counter. Build and program the PIC16F883 using your Lab 2 PORTB counter program and measure RB0's PW, PS, period & frequency capture an oscilloscope image showing enough of the ~~meas~~ waveform to verify the measurements. Compare PW & PS and explain why RB0 changes state at the observed rate based on the binary count & instructions in the main loop

AWD 9/8/26

Lab 3 Delays

AND 9/8/26



AND

Figure 3.1 Capture of RB0 standard counter waveform

TABLE 15-2: PIC16F882/883/884/886/887 INSTRUCTION SET

AND

Mnemonic, Operands		Description	Cycles	14-Bit Opcode MSb	LSb	Status Affected	Notes
BYTE-ORIENTED FILE REGISTER OPERATIONS							
ADDWF	f, d	Add W and f	1	00 0111 dfff ffff		C, DC, Z	1, 2
ANDWF	f, d	AND W with f	1	00 0101 dfff ffff		Z	1, 2
CLRF	f	Clear f	1	00 0001 1fff ffff		Z	2
CLRW	–	Clear W	1	00 0001 0xxx xxxx		Z	
COMF	f, d	Complement f	1	00 1001 dfff ffff		Z	1, 2
DECF	f, d	Decrement f	1	00 0011 dfff ffff		Z	1, 2
DECFSZ	f, d	Decrement f, Skip if 0	1(2)	00 1011 dfff ffff			1, 2, 3
INCF	f, d	Increment f	1	00 1010 dfff ffff		Z	1, 2
INCFSZ	f, d	Increment f, Skip if 0	1(2)	00 1111 dfff ffff			1, 2, 3
IORWF	f, d	Inclusive OR W with f	1	00 0100 dfff ffff		Z	1, 2
MOVF	f, d	Move f	1	00 1000 dfff ffff		Z	1, 2
MOVWF	f	Move W to f	1	00 0000 1fff ffff			
NOP	–	No Operation	1	00 0000 0xx0 0000			
RLF	f, d	Rotate Left f through Carry	1	00 1101 dfff ffff		C	1, 2
RRF	f, d	Rotate Right f through Carry	1	00 1100 dfff ffff		C	1, 2
SUBWF	f, d	Subtract W from f	1	00 0010 dfff ffff		C, DC, Z	1, 2
SWAPF	f, d	Swap nibbles in f	1	00 1110 dfff ffff			1, 2
XORWF	f, d	Exclusive OR W with f	1	00 0110 dfff ffff		Z	1, 2
BIT-ORIENTED FILE REGISTER OPERATIONS							
BCF	f, b	Bit Clear f	1	01 00bb bfff ffff			1, 2
BSF	f, b	Bit Set f	1	01 01bb bfff ffff			1, 2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff			3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff			3
LITERAL AND CONTROL OPERATIONS							
ADDLW	k	Add literal and W	1	11 111x kkkk kkkk		C, DC, Z	
ANDLW	k	AND literal with W	1	11 1001 kkkk kkkk		Z	
CALL	k	Call Subroutine	2	10 0kkk kkkk kkkk			
CLRWDT	–	Clear Watchdog Timer	1	00 0000 0110 0100		$\overline{TO}$, $\overline{PD}$	
GOTO	k	Go to address	2	10 1kkk kkkk kkkk			
IORLW	k	Inclusive OR literal with W	1	11 1000 kkkk kkkk		Z	
MOVLW	k	Move literal to W	1	11 00xx kkkk kkkk			
RETFIE	–	Return from interrupt	2	00 0000 0000 1001			
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk			
RETURN	–	Return from Subroutine	2	00 0000 0000 1000			
SLEEP	–	Go into Standby mode	1	00 0000 0110 0011		$\overline{TO}$, $\overline{PD}$	
SUBLW	k	Subtract k from literal	1	11 110x kkkk kkkk		C, DC, Z	
XORLW	k	Exclusive OR literal with W	1	11 1010 kkkk kkkk		Z	

Table 3.1 Instruction Set For PIC16F883

-See code block on pg 15 for main loop

2. Determine the instruction time. Use the measured waveform & instruction-cycle analysis of the main loop to calculate the actual instruction time.

AND 9/8/26

AND 9/8/26

Lab 3 Delays

If N instruction cycles occur durring one measured interval

$T_{cy} = t_{measured}/N$

$T_{cy} = 6ms/6$

$\boxed{T_{cy} = 1\mu s}$

3. Determine the oscillator freqaruency. Calculate the actual oscillator frequency from the measured instruction time

$F_{osc} = 4/T_{cy}$

$F_{osc} = 4/1\mu s$

$\boxed{F_{osc} = 4\,MHz}$

$$\frac{(|F_{measured} - F_{nominal}|)}{F_{nominal}} \cdot 100$$

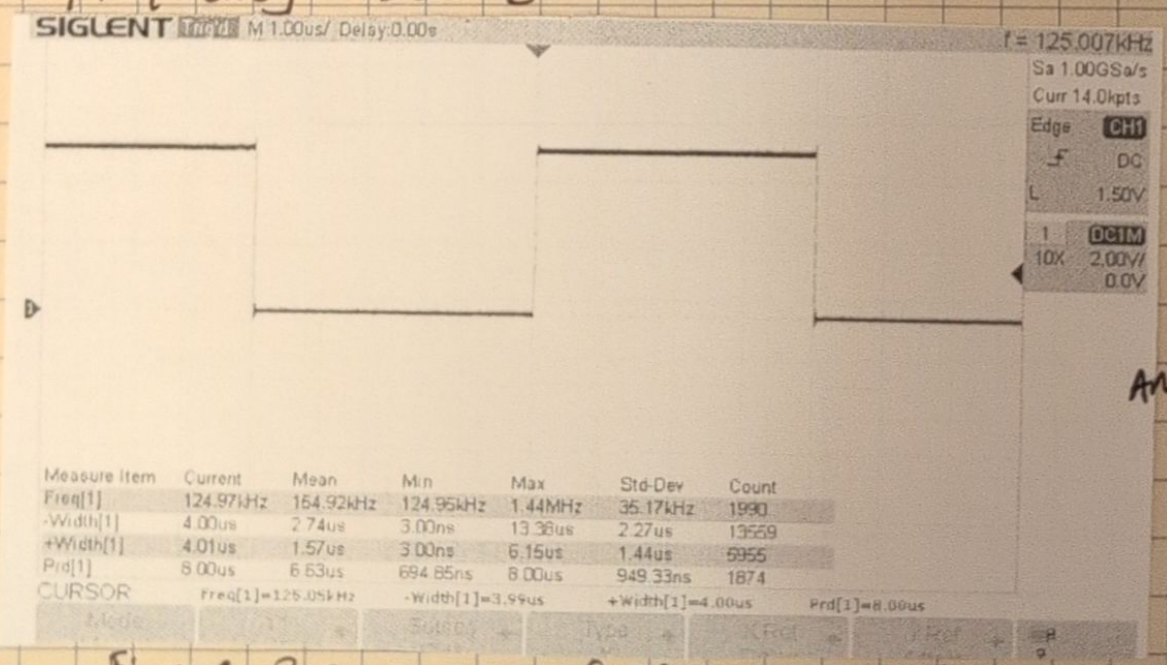
$[(4MHz - 4MHz)/4MHz] \cdot 100$

$\boxed{error\ \% = 0\%}$

Both predicted and measured waveforms came out to 4MHz

4. Add and remove NOP. Add one NOP instruction to the repeating exicution path of the main loop. Before programing the PIC predict the changes on RB0 PW, PS, Period, & Frequency. Build and program the modified code then measure RB0 again. Compare the measured timing differences

- Adding a NOP will add 1 cycle so the number of cycles per operations goes to 4 so using $T_{cy} = 1\mu s$ the PW & PS = $4\mu s$, Period = $8\mu s$, & Frequency = 125 kHz



AND 9/8/26

Figure 3.2 Capture of RB0 with NOP

Lab 3 Delays

Using a Universal counter on both setups I measured 166.673491 kHz frequency for INCF only, and 125.005091 kHz for INCF & NOP

TSR 9-8-26

$T_{cy\,INCF} = t_{measured}/N$

$T_{cy\,INCF} = 166.673491\,kHz^{-1}/6$

$\boxed{T_{cy\,INCF} = 999.959\,ns}$

$T_{cy\,INCF\&NOP} = t_{measured}/N$

$T_{cy\,INCF\&NOP} = 125.005091\,kHz^{-1}/8$

$\boxed{T_{cy\,INCF\&NOP} = 999.959\,ns}$

$Fosc = 4/999.959\,ns$ $Fosc = 4.000164\,MHz$

$error\% = (|F_{measured} - F_{nominal}|)/F_{nominal} \cdot 100$

$error\% = (|4.000164\,MHz - 4\,MHz|)/4\,MHz \cdot 100$

$\boxed{error\% = 0.0041\%}$

Part 2 - Create a Precise Short Delay:

Goal: Create a new PIC16F883 project that toggles RB0 using XORWF, then add a calculated software delay to produce a 10kHz square wave.

Before Lab:

Create a new MPLAB X project for this part. Use the project structure, PIC configuration, oscillator configuration, and assembly conventions established in Lab 02. Reference earlier documentation where nothing has changed. Begin with this repeating output loop:

```
Main Loop:
    MOVLW 0x01  ; 1Tcy
    XORWF PORTB,1 ; 1Tcy
    GOTO Main Loop ; 2Tcy
```

Determine what waveform this program should produce on RB0, use $T_{cy} = 1\,\mu s$, predict: PW, PS, period, frequency, and Duty cycle

$\boxed{PW = 4\,\mu s}$ $\boxed{PS = 4\,\mu s}$ $\boxed{period = 8\,\mu s}$ $\boxed{frequency = 125\,kHz}$

$\boxed{DC = 50\%}$

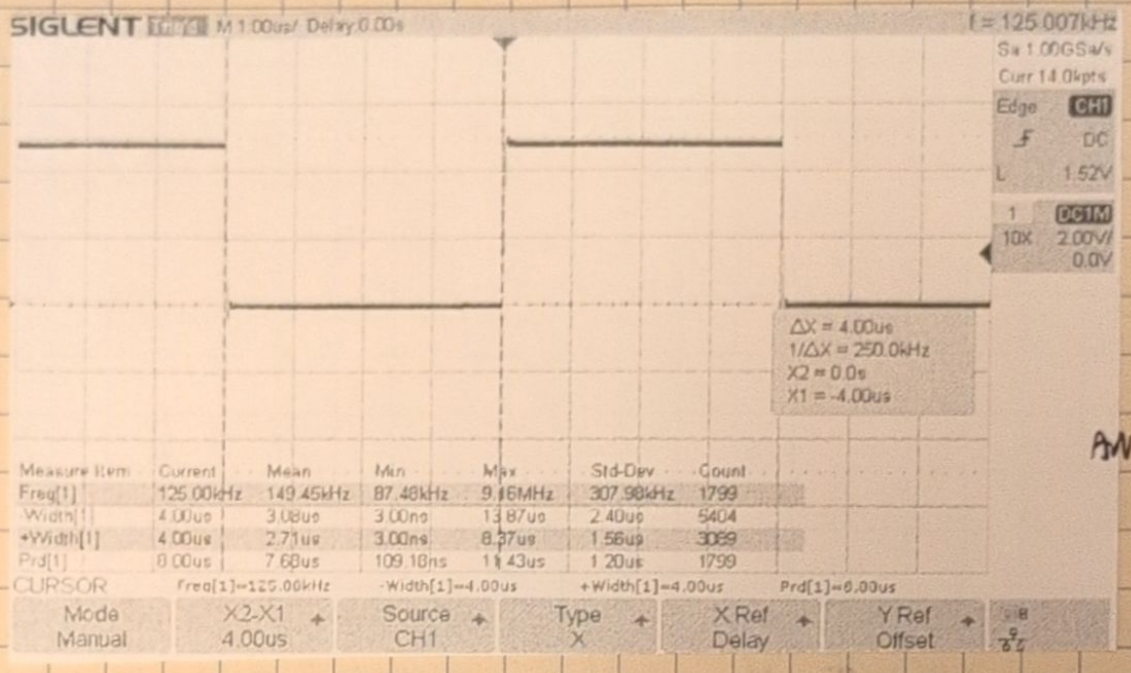
AND 9/8/26

AND 9/9/26

Lab 3 Delays

In the Lab:

1. Verify the XORWF toggle loop. Measure RB0 using the oscilloscope & frequency counter, and record: PW, PS, Period, frequency, and Duty cycle



AND

Figure 3.3 Capture of RB0 XORWF measurements

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter (AND)
PW	4 µs	3.999 836 µs	4 µs	4.008 µs
PS	4 µs	3.999 836 µs	4 µs	3.992 µs
Period	8 µs	7.999 672 µs	8 µs	7.999 673 µs
Frequency	125 kHz	125.005 125 kHz	125 kHz	125.00509 kHz
Duty cycle	50%	50%	50.01%	50.1%

Table 3.2 Comparison table of RB0 measurements

XORWF compares PORTB with W=0x01 and then stores the result back in to RB0 so on every cycle it only alternates RB0 and not RB1-RB7.

2. Determine the required timing, Modify the program so RB0 produces a 10kHz 50% DC signal

AND 9/9/26

period = 1/10kHz = 100µs | PW=PS = 50µs | Tcy = 1µs

Lab 3 Delays

AND 9/9/26

With each Pulse Width & Pulse Space needing 50μs and the current code allows for 4μs there needs to be an additional 46 TCy added to the code so each PW & PS = 50μs

3. Create a Software delay loop, use an 8 bit counter and DECFSZ to add the required delay

```
    count    EQU 0x20                ;use data memory address 0x20 as an 8-bit loop counter

MainLoop:
    MOVLW 0x01                       ;1 TCY code not part of delay
    XORWF PORTB,1                    ;1 TCY code not part of delay

    MOVLW 0x09                       ;1 TCY delay overhead
    MOVWF count                      ;1 TCY delay overhead
DelayLoop:
    DECFSZ count,1                   ;1(2) 1 TCY if not zero, 2 TCY if zero
    GOTO DelayLoop                   ;2 TCY

    GOTO   MainLoop                  ;2 TCY code not part of delay
```

AND

Figure 3.4 Example Code for Delay

$T_{Loop} = 1+1+1+1+N(1+2)-1+2$

$T_{Loop} = 4+(9)(1+2)-1+2$

~~$T_{Loop} = 30$~~ $T_{Loop} = 32$

The PW & PS for the code above is 32μs

4. Determine the counter value to have 50μs PW&PS, then determine the instruction cycles consumed outside the delay, instruction cycles required in the delay, the required counter value, & ~~width~~ whether any NOP instructions are needed to achive the desired timing

$50 = 3N+5$

$45 = 3N$

$N = 15$

The Delay will cycle 15 times for a 50μs pulse width & space with no additional NOP needed

Compared to the example code only the count has changed

```
Main:
    count    EQU 0x20                ;use data memory address 0x20 as an 8-bit loop counter

MainLoop:
    MOVLW 0x01                       ;1 TCY code not part of delay
    XORWF PORTB,1                    ;1 TCY code not part of delay

    MOVLW 0x0F                       ;1 TCY delay overhead
    MOVWF count                      ;1 TCY delay overhead

DelayLoop:
    DECFSZ count,1                   ;1(2) 1 TCY if not zero, 2 TCY if zero
    goto DelayLoop                   ;2 TCY
    goto MainLoop                    ;2 TCY code not part of delay
End
```

AND

Figure 3.5 Delay Code for 50μs PW&PS

AN 09/9/26

36

AND 9/9/26

Lab 3 Delays

5. Build & measure the calculated delay

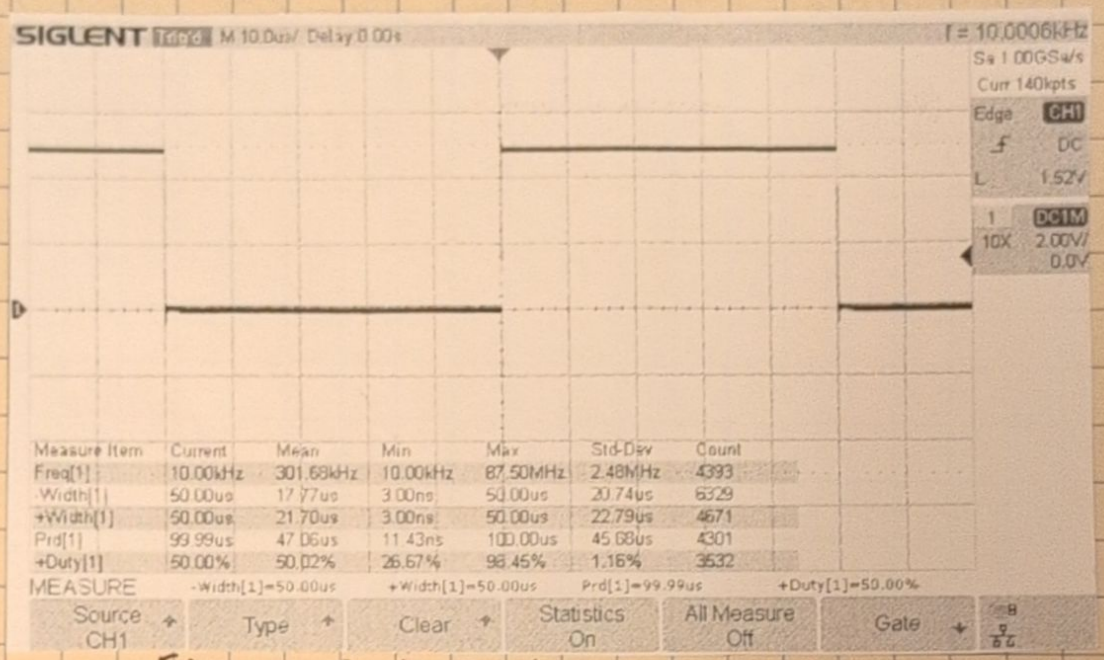


Figure 3.4 Capture of RB0 10kHz

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	50µs	49.99795µs	50µs	50.006µs
PS	50µs	49.99795µs	50µs	49.989µs
Period	100µs	99.9959µs	99.99µs	99.99588µs
Frequency	10KHz	10.00041kHz	10KHz	10.00041kHz
Duty cycle	50%	50%	50%	50.008%

JJR 9-9-26

Table 3.3 Comparison table for RB0 10kHz

Part 3 Create & verify a long Delay:

Goal: Use nested loops to create a longer delay that can be produced with a single 8-Bit loop. Design a program that alternates a DLG 7137 display between 0 & 5 with a PW & PS = 0.50000 Seconds, then determine whether available equipment is able to verify that level of accuracy

Before Lab:

Continue using Tcy = 1µs & Determine the longest delay that can be produced using a single 8-Bit loop from part 2 and explain why that method is not practical using a 0.5 Sec delay. then Design an inline nested delay loop capable of producing a 0.5 Sec delay.

AND 9/9/26

for the single 8-Bit loop the absolute maximum time you could have is: AND 9/10/26

$3N + 5 = Tcy$
$3(256) + 5 = Tcy$
$\boxed{773 = Tcy}$

By setting the inital count to 0 the decrement cycles back to the max value of 255 so 256 total values for the count.

With only 773 cycles one 8-Bit loop produces a 773 µs delay which is about 650 times too small.

```
Main:
    MOVLW 0x05          ;1 TCY
    XORWF 0x07,1        ;1 TCY

    MOVLW 0X04          ;1 TCY
    MOVWF outCount      ;1 TCY
    goto OutDelay       ;2 TCY

OutDelay:
    MOVLW 0XC0          ;1 TCY
    MOVWF midCount      ;1 TCY
    goto MidDelay       ;2 TCY

MidDelay:
    MOVLW 0xD7          ;1 TCY
    MOVWF inCount       ;1 TCY
    goto InDelay        ;2 TCY

InDelay:
    DECFSZ inCount      ;1 (2) TCY
    goto InDelay        ;2 TCY

    DECFSZ midCount     ;1 (2) TCY
    goto MidDelay       ;2 TCY

    DECFSZ outCount     ;1 (2) TCY
    goto OutDelay       ;2 TCY
    goto Main           ;2 TCY
End
```

AND

Figure 3.7 Code Block for 0.5sec Delay using Nested loops

$I_3 - 1$
$I_3 + 3$
$I_3 + 6$
$M(3I + 6) + 6$
$O[M(I_3 + 6) + 6] - 1$
$\boxed{O[M(I_3 + 6) + 6] + 7}$

O = Outer Delay Loop
M = Mid Delay Loop
I = inDelay Loop

When calculating for a 0.5sec delay the closest I got was by using

O = 4 M = 192 I = 215

$4[192(215 \cdot 3 + 6) + 6] + 7 = Tcy$
$Tcy = 499,999$

By adding a single NOP at the begining of the code before the loop it will be 500,000 Tcy = 0.5sec when Tcy = 1µs. So by using this Pw & PS = 0.5sec, period = 1sec frequency = 1Hz, & DC = 50%. Using the measured Tcy from part 1 Pw & PS = ~~499~~ 0.4999795 sec, Period = 0.999959sec frequency = 1.000041Hz, and DC = 50%

AND 9/10/26

Lab 3 Delays

AND 9/10/26

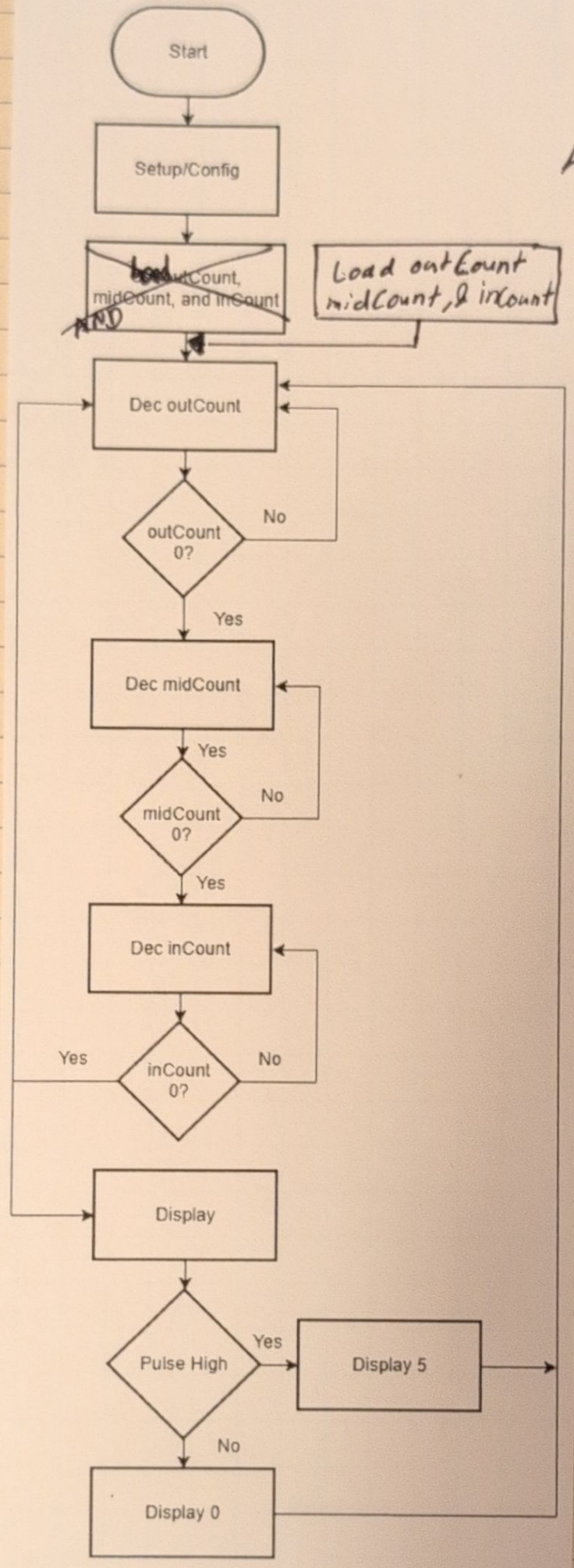


Figure 3.8 Flow chart for 0.5sec delay

Lab 3 Delays

AND 9/10/26

In the Lab:

1. Build the nested delay using the calculated values, the DIG 7137 must alternate between 5 & 0 every 0.5 sec. Show any modified code

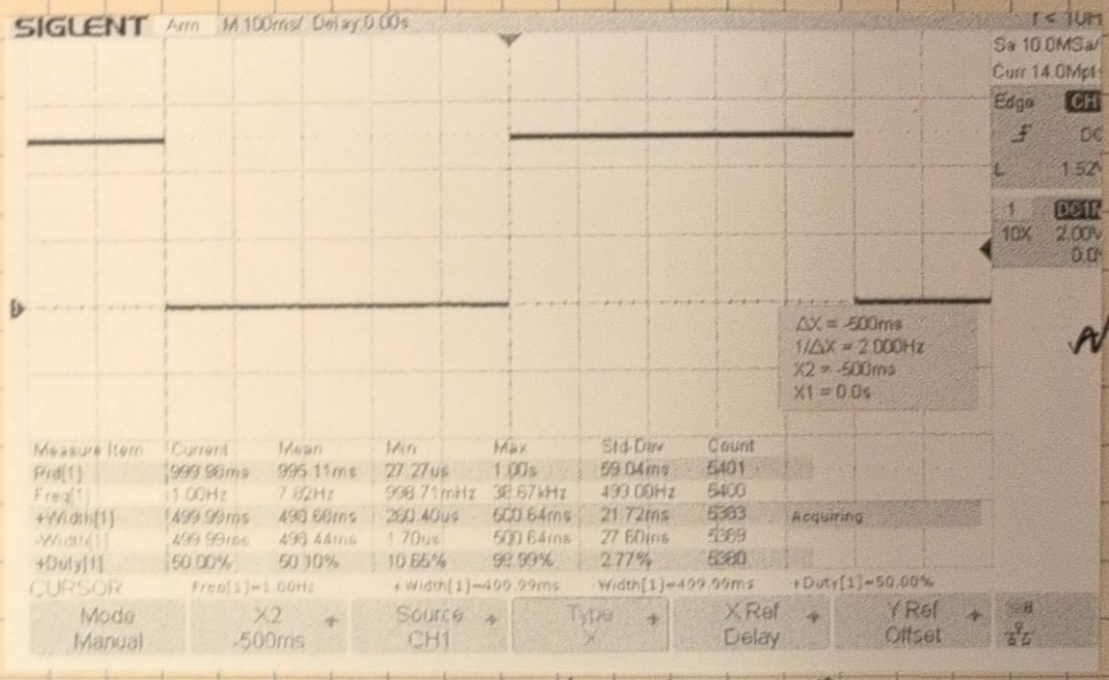
```
Main:
    MOVLW 0x05      ;1 TCY
    XORWF 0x07,1    ;1 TCY

    MOVLW 0X04      ;1 TCY
    MOVWF outCount  ;1 TCY
    goto OutDelay   ;2 TCY

OutDelay:
    MOVLW 0XC0      ;1 TCY 192 in decimal
    MOVWF midCount  ;1 TCY
    NOP
    NOP
    NOP
    NOP
    NOP
    goto MidDelay       ;2 TCY

MidDelay:
    MOVLW 0xD7          ;1 TCY 215 in decimal
    MOVWF inCount       ;1 TCY
    goto InDelay        ;2 TCY

InDelay:
    DECFSZ inCount      ;1 (2) TCY
    goto InDelay        ;2 TCY

    DECFSZ midCount     ;1 (2) TCY
    goto MidDelay       ;2 TCY

    DECFSZ outCount     ;1 (2) TCY
    goto OutDelay       ;2 TCY
    goto Main           ;2 TCY

End
```

AND

Added NOP were needed to achive a 0.5sec with ± 1ms accuracy

Figure 3.9 Modified 0.5 sec Nested Loop Delay code block

2. Measure with oscilloscope and check PW, PS, Period, frequency, & Duty Cycle



AND

Figure 3.10 Capture of 0.5S Nested Loop Delay

AND 9/10/26

AND 9/10/26

Lab 3 Delays

Because each cursor movement is 2ms we can't determine if the delay is within ±1μs from 0.5 sec

3. Measure with the frequency counter and record PW, PS, period, frequency, & DC

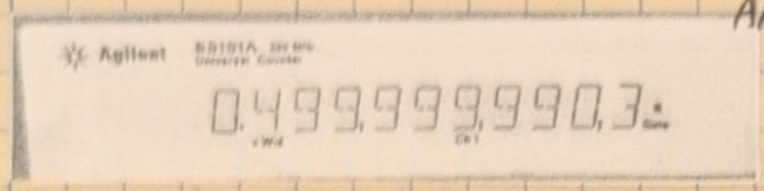


Figure 3.11 capture of PW on FC

AND

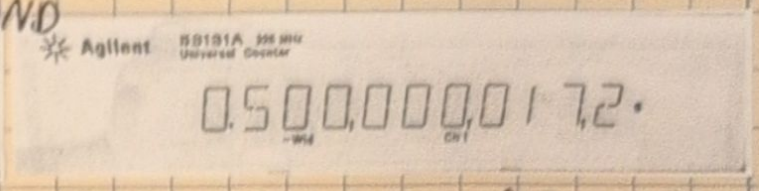


Figure 3.12 Capture of PS on FC

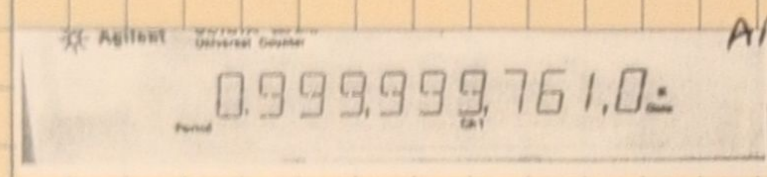


Figure 3.13 capture of period on FC

AND

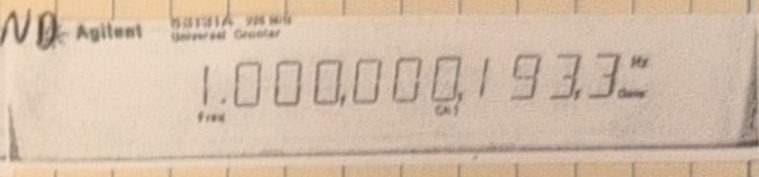


Figure 3.14 Capture of frequency on FC

Figure 3.15 Capture of DC on FC

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	0.5 sec	0.4999759 sec	0.49999 sec	0.499999903sec
PS	0.5 sec	0.4999759 sec	0.49999 sec	0.500000172sec
Period	1 sec	0.999959 sec	0.99998 sec	0.999999761sec
Frequency	1 Hz	1.00004 Hz	1.00 Hz	1.000001933Hz
Duty cycle	50%	50%	50%	50.000003%

Table 3.4 Comparison Table for 0.5sec Delay

TJR 9-11-26

4. Add one NOP to the exicution path between transistions and predict Pw, PS, period, and Frequency, then measure PW

By adding one NOP PW & PS = 0.500001 sec, Period = 1.000002 sec, & frequency = 0.999998 Hz

Measurement	Before NOP	With NOP	Difference	Instrument Resolution AND
PW	0.5 sec	0.500000912	0.000000912	100 ps

AND 9/10/26

Part 4 - Delay Subroutines & the Hardware Stack:

Goal: Refactor the working inline delay from part 3 into reusable subroutines. Use CALL & RETURN, including a nested subroutine call, and verify how the new program structure affects execution timing & the hardware return stack.

Before Lab:

Modify the part 3 program so it contains:

- a main program that alternates the OLG7137 between 0 & 5
- a reusable long delay subroutine
- a shorter delay subroutine called from within the long-Delay subroutine
- appropriate CALL & RETURN instructions

```
Main:
    ; main code goes here
    MOVLW 0x05 ;1 TCY
    XORWF 0x07,1 ;1TCY
    CALL outDelay ;2TCY
    goto Main ;2TCY

    outDelay:
    MOVLW 0x04 ;1 TCY
    MOVWF outCount ;1 TCY

    outLoop:
    CALL midDelay ;2 TCY
    DECFSZ outCount; 1 (2) TCY
    goto outLoop ;2 TCY
    RETURN ;2 TCY

    midDelay:
    MOVLW 0xB7 ;1 TCY
    MOVWF midCount ;1 TCY

    midLoop:
    CALL inDelay ;2 TCY
    DECFSZ midCount ;1 (2) TCY
    goto midLoop ;2 TCY
    RETURN ;2 TCY

    inDelay:
    MOVLW 0xE1 ;1 TCY
    MOVWF inCount ;1 TCY

    inLoop:
    DECFSZ inCount ;1 (2) TCY
    goto inLoop ;2 TCY
    RETURN ;2 TCY

End
```

Figure 3.16 Code Block for 0.5sec delay using CALL functions

AND

Because the structure is similar to part 3 the equation for TCY only changes slightly due to each CALL (2TCY) also needing a RETURN which is also 2TCY compared to just goto which is only 2TCY

$$Tcy = O[M(I3+8)+8]+9$$

When calculating the closest values I found were
$O=4$ $M=183$ $I=225$

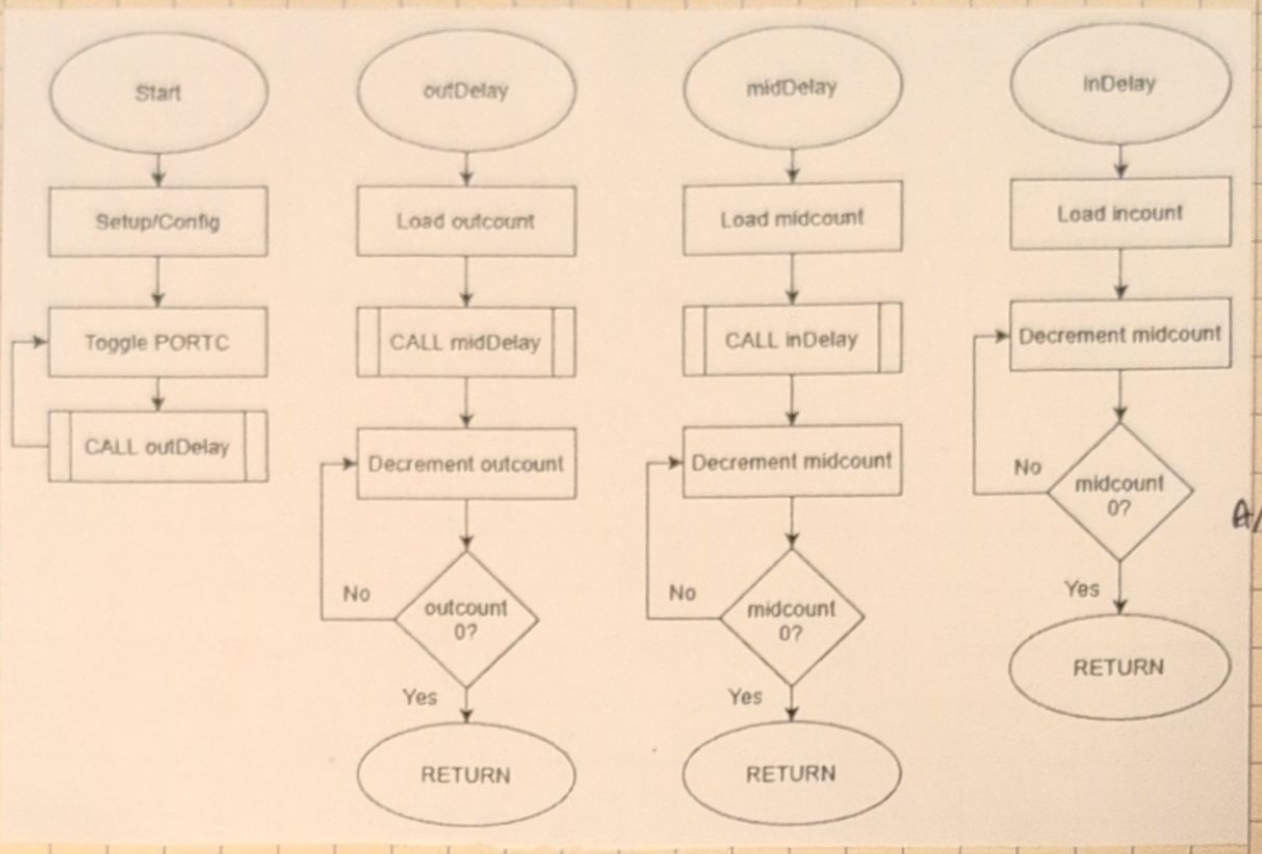
$$Tcy = 4[183(225\cdot3+8)+8]+9$$
$$Tcy = 499,997$$

With these values only 3 NOP should theoretically be needed

AND 9/11/26

AND 9/11/26

Lab 3 Delays



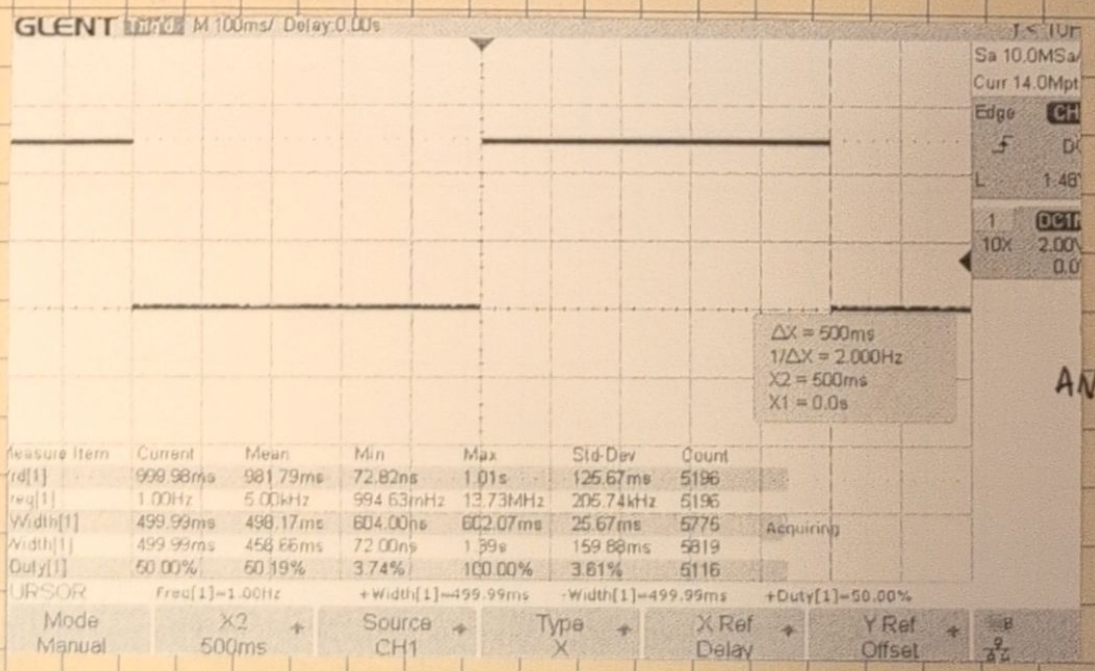
AND

Figure 3.17 Flowchart for 0.5 sec delay using CALL functions

In the lab:

1. Build and program the refactored code & verify the DLG 7137 coninuously alternates between 0 & 5

2. Measure PW, PS, period, frequency & Duty Cycle with an oscilloscope & frequency Couter, Then compare to nominal prediction using tcy = 1 µs and measured tcy from Part 1



AND

AND 9/11/26

Figure 3.18 Capture of 0.5 sec CALL Delay

~~LAB 3~~ Lab 3 Delays

AND 9/11/26

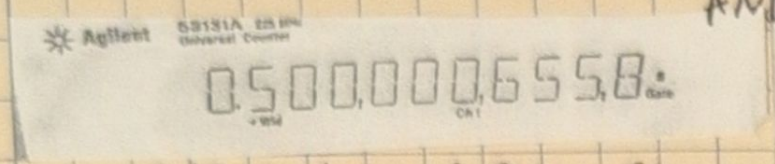


Figure 3.19 Capture of PW on FC

AND

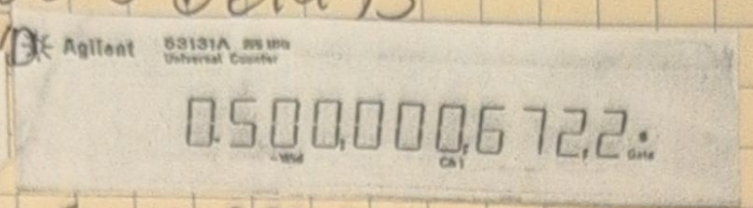


Figure 3.20 Capture of PS on FC

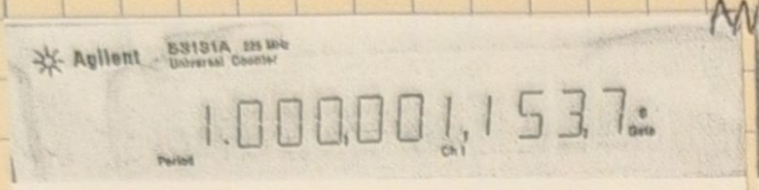


Figure 3.21 Capture of Period on FC

AND

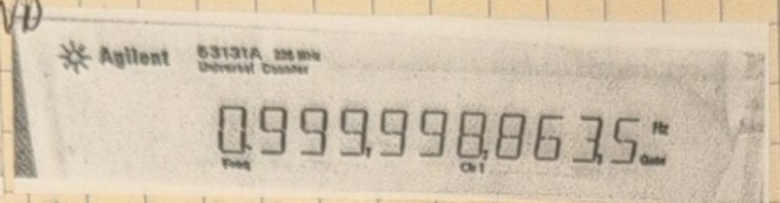


Figure 3.22 Capture of frequency on FC

AND

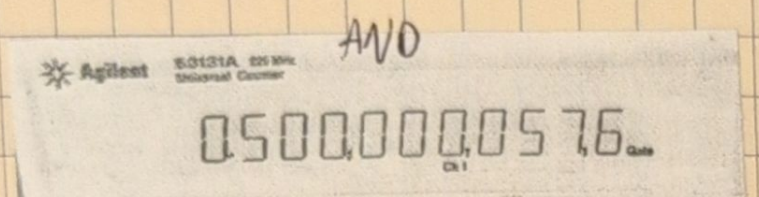


Figure 3.23 Capture of DC on FC

AND

```
Main:
    ; main code goes here
    MOVLW 0x05 ;1 TCY
    XORWF 0x07,1 ;1TCY
    CALL outDelay ;2TCY
    goto Main ;2TCY

    outDelay:
    MOVLW 0x04 ;1 TCY
    MOVWF outCount ;1 TCY

    outLoop:
    CALL midDelay ;2 TCY
    DECFSZ outCount; 1 (2) TCY
    goto outLoop ;2 TCY
    NOP
    NOP
    NOP
    RETURN ;2 TCY

    midDelay:
    MOVLW 0xB7 ;1 TCY
    MOVWF midCount ;1 TCY

    midLoop:
    CALL inDelay ;2 TCY
    DECFSZ midCount ;1 (2) TCY
    goto midLoop ;2 TCY
    NOP
    NOP
    NOP
    NOP
    NOP
    RETURN ;2 TCY

    inDelay:
    MOVLW 0xE1 ;1 TCY
    MOVWF inCount ;1 TCY

    inLoop:
    DECFSZ inCount ;1 (2) TCY
    goto inLoop ;2 TCY
    RETURN ;2 TCY

End
```

Added NOPs were used to get within ±1µs for the Delay

Figure 3.24 Full code block for 0.5 sec delay using CALL

AND 9/11/26

lab 3 Delays

AND 9/11/26

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter AND
PW	0.5sec	0.49999759sec	0.49999sec	0.500000558sec
PS	0.5sec	0.49999759sec	0.49999sec	0.500000722sec
Period	1sec	0.999959sec	0.99998sec	1.000001537sec
Frequency	1Hz	1.00004Hz	1.00Hz	0.999998635sec
Duty cycle	50%	50%	50%	50.00000576sec

Table 3.5 comparison table for 0.5sec delay with CALL

TSE 9-11-26

3. Trace the nested calls

Main:
→~~out~~
→CALL out Delay
→CALL mid Delay
→CALL in Delay
← RETURN
← RETURN
← RETURN
Main Continues

Conclusion:

In this lab I created & used different types of delays in order to make a DLG 7137 display alternate between 0 & .5. In part 1 I tested the accuracy of the instructions cycles & whether or not it was acctually 1μs per Tcy but after measuring it was 999.959ns (see pg 33). In part 2 after changing the code to include a delay of 50μs I remeasured PW, PS, period, frequency, & DC (see pgs 35 & 36 for calculations & measurements). In part 3 I used nested loops to create a 0.5 sec delay (see pgs 37, 39, & 40 for calculations & measurements). Part 4 adjusted the code from nested loops to CALL functions at a 0.5 sec delay as well.

AND 9/11/26